AGLA STATION

Knowledge Document

A Telegram assistant for the Delhi NCR metro: product, architecture, system design, operations and how it was built

Item	Detail
Document	Agla Station knowledge document, version 1.0
Date	24 October 2026
Status	Final draft for review
Owner	Tzing66 (GitHub)
Repository	github.com/Tzing66/agla-station (public)
Audience	Engineers, reviewers and stakeholders who need to understand, run or extend the system
Try the bot	t.me/agla_station_bot (Telegram)
Demo video	youtu.be/NhArOnjwbjg
Companion documents	README.md (running it), docs/architecture.md (as-built design), docs/guide.md (file-by-file guide), ledger/decisions.md (decision log), docs/build-journal.html (build timeline)

Contents

1. Executive summary
2. Product overview
3. System architecture
4. Key flows
5. Data and fares
6. Domain logic
7. Interfaces
8. AI design
9. Security and privacy
10. Quality engineering
11. Operations
12. How it was built
13. Limitations, risks and roadmap
Appendix A. Glossary

Figures: logical architecture, runtime, trip booking sequence, message decision flow, sharing and arrival sequence, data pipeline, fare calculation, entity relationships, development process.

1. Executive summary

Agla Station is a Telegram bot for the whole Delhi NCR metro network: DMRC lines, the Airport Express, Rapid Metro Gurugram and the Noida Aqua Line. A rider types where they are going in plain English, Hinglish or Hindi; the bot plans the route, quotes the official fare, provides a UPI payment link, issues a demo QR ticket, and rewards riding with Metro Miles, badges and a monthly Metro Wrapped. Riders can share a live journey link and send an "I've reached" message. A Telegram Mini App dashboard shows their map, badges, miles, insights and Wrapped.

The system is a single Python process (Telegram bot plus FastAPI web app) over SQLite, with an independent MCP server exposing the same metro tools to Claude Code and Claude Desktop. Claude Sonnet 5.5 understands rider messages and picks tools; every number a rider sees is computed in Python.

Key facts

Area	Status
Network coverage	275 stations on 14 lines and branches across four networks (DMRC, Airport Express, Rapid Metro, Aqua Line)
Fare accuracy	405 of 405 official DMRC journey-planner fares reproduced; one documented planner quirk
Quality	174 automated tests; CI on every push (lint, tests, secret scan of full history)
AI safety	Number guard on every model reply; the model cannot set amounts, log trips or redeem rewards
Language understanding	Station-parsing eval: 50/50 for both Claude Sonnet 5.5 and Claude Haiku 5.5
Build	Built in Claude Code over 21.3 hours (7.9 active): 131 human prompts, 876 Claude actions, 6 logged decisions

Design principles

- **Code decides, the LLM narrates.** Routes, fares, times, miles, badges and statistics are computed in Python; the model only chooses tools and writes words, and a number guard rejects any reply containing a number that no tool produced.
- **One tool layer.** All metro logic lives in agla/tools; the bot, web app and MCP server are thin front ends. Automated tests enforce this boundary.
- **Money stays in code.** Payments reference a stored quote, so the amount always comes from code; "I've paid", "I've reached" and redemption are buttons, never model actions.
- **Never invent fares.** Fares come from official sources recorded with provenance and are verified against official answers in the test suite.
- **Privacy by default.** Share links expire and show only a first name; the bot never messages anyone except the rider.
- **Honest constraints.** Tickets are mock QR tickets (DMRC has no public ticketing API), payment is an unverified UPI intent link, journey position is a timetable estimate rather than GPS, and rewards are a concept demo.

2. Product overview

2.1 Problem and users

Delhi NCR's metro spans several operators with different fare rules, and riders describe journeys the way they speak ("RC se botanical garden, sasta wala"). Agla Station gives commuters one conversational place to plan, pay for and share a trip, and adds light gamification to make regular riding rewarding. The name comes from the in-train announcement "agla station…" ("next station").

2.2 Capabilities

Capability	What the rider gets
Route planning	Fastest, fewest changes or cheapest route from free-text station names, typos, abbreviations, landmarks and Hinglish; asks when a name is ambiguous
Fares and payment	Official token fare (regular Monday to Saturday, discounted on Sundays and national holidays), UPI intent link and QR for GPay, PhonePe or Paytm
Tickets	Demo QR ticket image after "I've paid" (clearly marked as a demo)
Journey sharing	A live page with the traveller's estimated position and ETA, sent through Telegram's own share sheet
I've reached	A ready-made arrival message; asks "Are you sure?" if tapped well before the ETA
Metro Miles and rewards	Miles per km (double off-peak) plus bonuses; a demo rewards catalogue and vouchers
Badges	11 achievement badges plus a Line Collector badge for every line
Metro Wrapped	A monthly recap as a three-stop "ride" in the Mini App and a shareable card in chat
Dashboard Mini App	Map of visited stations, line progress, badges, miles and redemption, weekly insights, Wrapped archive
Languages	English, Hinglish or Hindi replies, chosen per rider

2.3 Scope and non-goals

- In scope: the four NCR metro operators, token fares, a single-process deployment suitable for demos and small groups.
- Not in scope: real ticket issuance or payment verification (no public APIs), live train positions, smart-card discounts, group trip planning (parked in the backlog).

3. System architecture

3.1 Logical architecture

The system is organised in layers. Front ends handle transport and presentation only; the tool layer is the single public API for all metro logic; domain modules implement miles, badges, sharing and Wrapped; the network package loads the metro graph; and agla/db.py owns the schema.

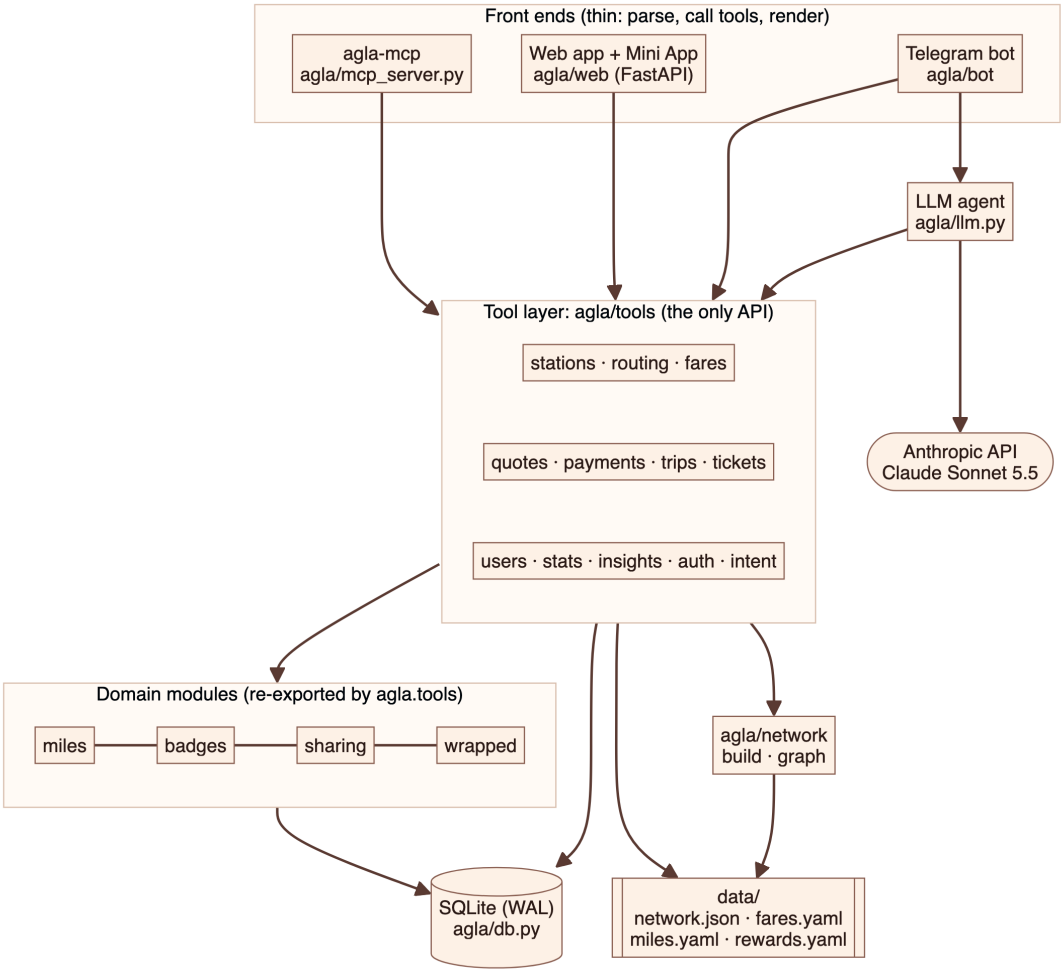


Figure 1. Logical architecture: thin front ends over a single tool layer

Component	Responsibility
Telegram bot (agla/bot)	Handlers, button menu, message templates, per-rider rate limit, off-topic policy
LLM agent (agla/llm.py)	Claude tool-use loop, prompt caching, number guard, Wrapped narration
Web app (agla/web)	Public share pages; dashboard Mini App and its JSON API; health check
agla-mcp (agla/mcp_server.py)	Seven read-only metro tools over MCP (stdio) for Claude Code and Claude Desktop
Tool layer (agla/tools)	Stations, routing, fares, quotes, payments, trips, tickets, users, stats, insights, auth, intent
Domain modules	miles.py, badges.py, sharing.py, wrapped.py; run as post-trip hooks or on demand

Component	Responsibility
Network (agla/network)	Builds network.json from official snapshots; loads a networkx routing graph
Storage (agla/db.py)	SQLite schema, migrations (version 4), WAL mode, foreign keys

3.2 Runtime and deployment

One process runs both the Telegram bot (long polling) and the web app (uvicorn) on a single asyncio event loop with one SQLite connection; web handlers are asynchronous so the connection is never shared across threads. With --tunnel, the process starts a Cloudflare quick tunnel for a public HTTPS address, which Telegram requires for Mini Apps. A watchdog probes the public address every 30 seconds and replaces the tunnel after three failed probes, then points Telegram's chat menu button at the new address. scripts/run.sh keeps the host awake and restarts the process if it exits.

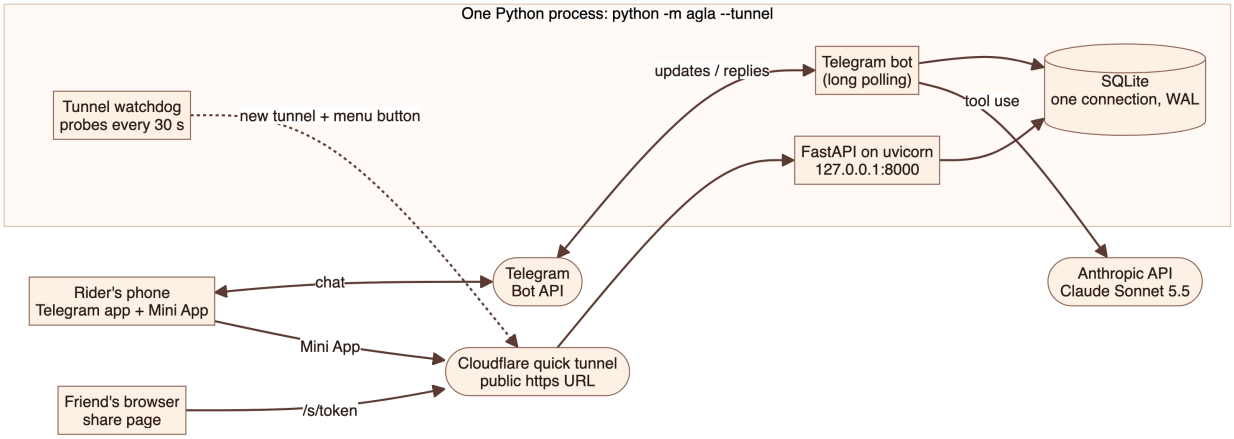


Figure 2. Runtime view: one process, Telegram long polling, Cloudflare tunnel for the web

> **Deployment note**
> The current deployment targets a laptop for demonstrations: quick-tunnel URLs change on restart, so earlier share links stop working. A fixed host (for example a small cloud VM with a named tunnel) removes this limitation without code changes beyond configuration.

4. Key flows

4.1 Planning and buying a trip

A rider's message passes through cost controls before reaching Claude. Claude calls quote_trip; the tool layer resolves stations, plans the route, prices it and stores a quote with a 30-minute expiry. The model receives route details without any numbers or the quote id, so it cannot restate or alter them. The bot renders the route card, payment link and "I've paid" button from the stored quote. Logging the trip runs the post-trip hooks (visits, miles, badges) in one transaction.

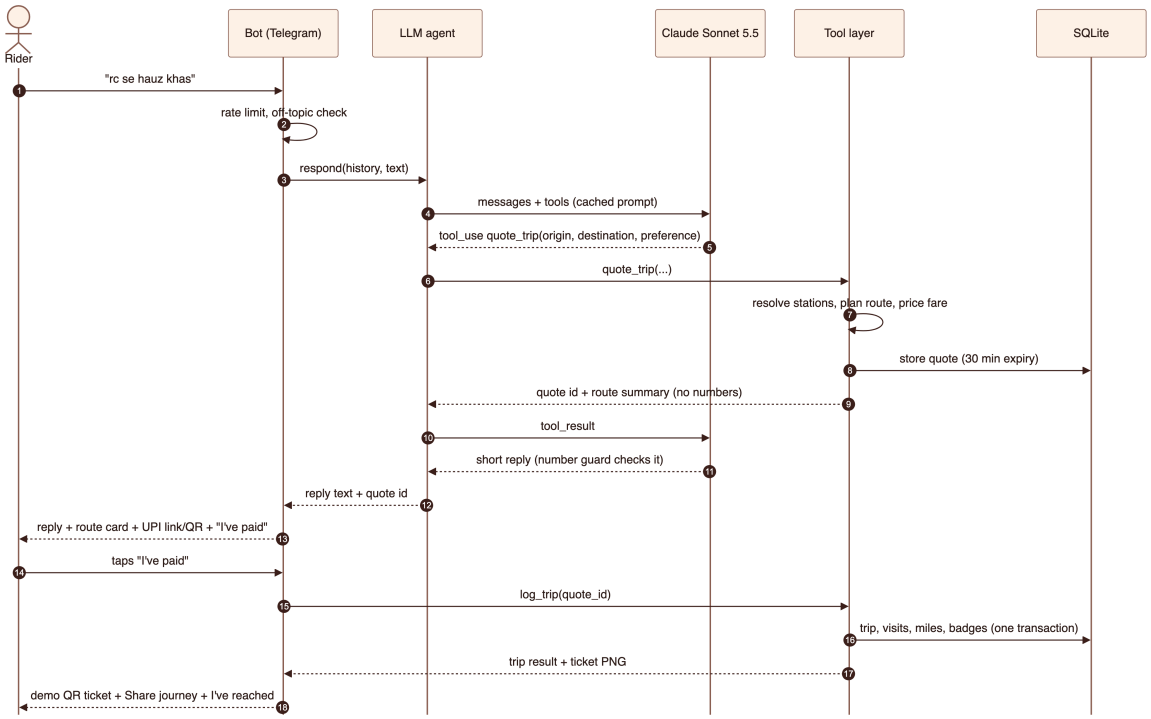


Figure 3. Sequence: from a free-text request to a ticket

4.2 Model turn, cost controls and the number guard

Menu buttons and repeated off-topic chatter are answered without calling the model. Messages that do reach Claude are rate-limited per rider, capped at four tool rounds and carry at most six messages of history. Every reply passes the number guard before it is sent; a reply that fails is replaced by a template.

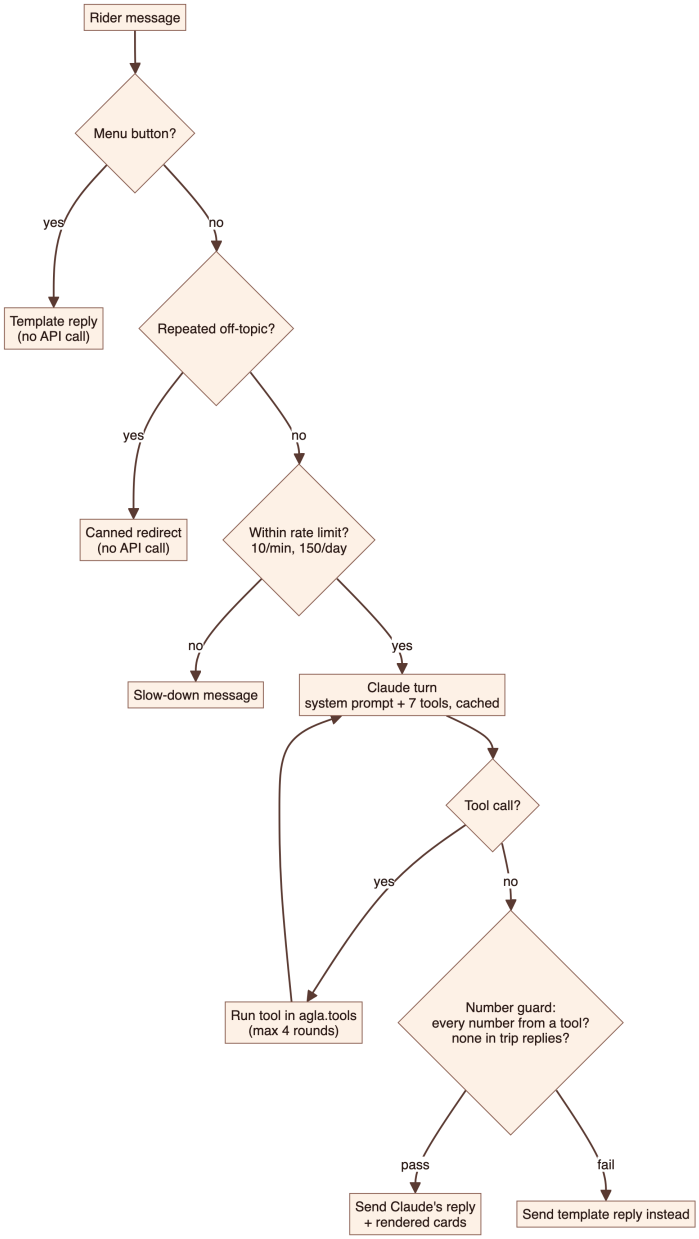


Figure 4. Decision flow for one rider message

4.3 Journey sharing and arrival

Sharing creates an expiring token. The rider chooses the recipient in Telegram's share sheet; the bot never messages third parties. The share page refreshes every 30 seconds with a timetable-based position estimate. "I've reached" marks the trip arrived, flips the share page to Arrived and shortens the link's life to 30 minutes. Because position is an estimate, an early tap asks the rider to confirm instead of refusing; an early confirmation counts but earns no arrival miles.

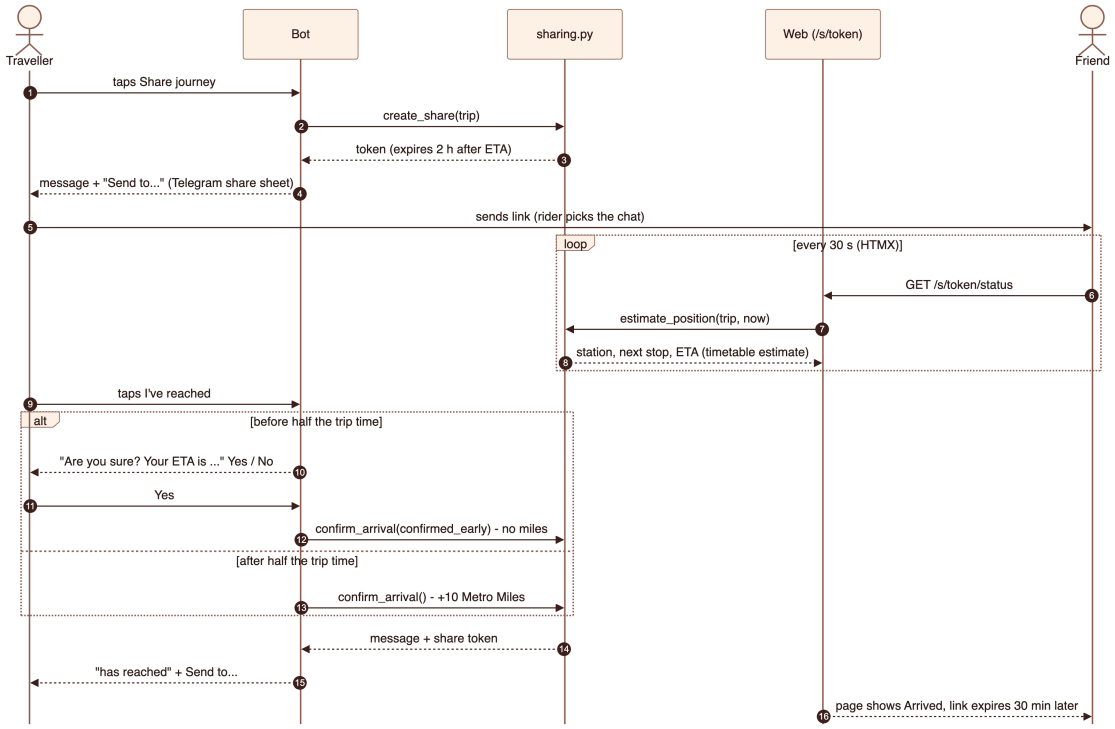


Figure 5. Sequence: sharing a journey and confirming arrival

5. Data and fares

5.1 Network data pipeline

The published DMRC GTFS feed was unavailable (its download returned 404), so the network was built from one-time snapshots of the official DMRC journey-planner API, the NMRC planner and fare table, and OpenStreetMap coordinates, read through a real browser because the API rejects non-browser clients. Hand-made fixes (aliases, walking links, one-way loops) live in data/manual with their provenance. The build is reproducible and the app makes no external calls at runtime.

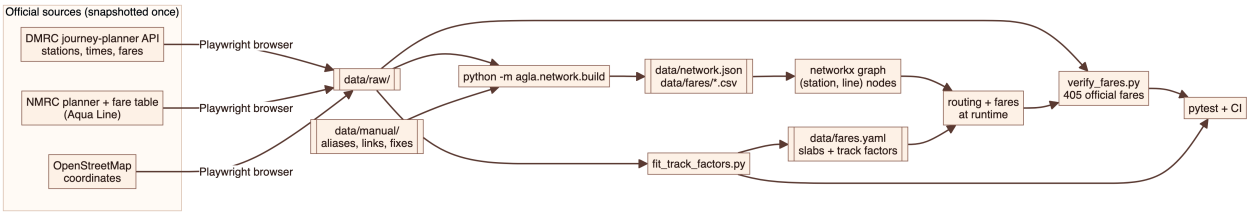


Figure 6. Data pipeline from official sources to runtime and verification

5.2 Fare model

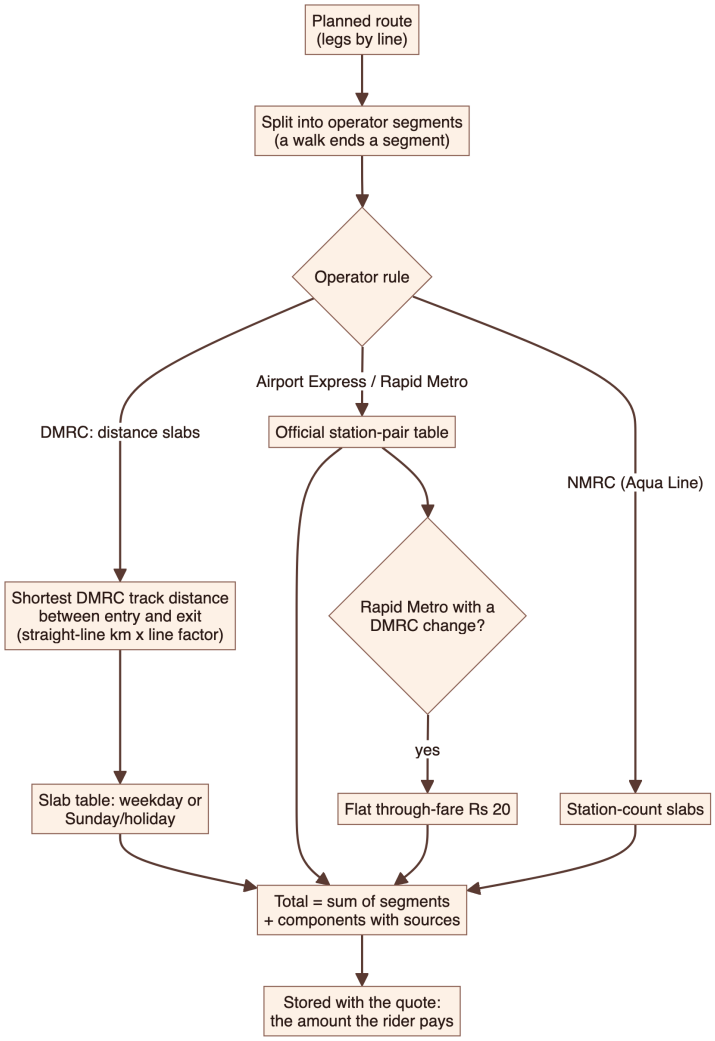


Figure 7. Fare calculation

Operator	Rule	Source
DMRC (most lines)	Distance slabs, Monday to Saturday: up to 2 km Rs 11, 5 km Rs 21, 12 km Rs 32, 21 km Rs 43, 32 km Rs 54, beyond Rs 64. Sunday/holiday: Rs 11, 11, 21, 32, 43, 54. Charged on the shortest track distance between entry and exit.	DMRC fare revision (Aug 2025), verified against the journey planner
Airport Express	Official station-pair table; no Sunday discount	DMRC journey planner, every pair
Rapid Metro Gurugram	Official station-pair table; flat Rs 20 when the trip changes to or from DMRC at Sikanderpur	DMRC journey planner, every pair
NMRC Aqua Line	Station-count slabs, Monday to Saturday Rs 10 to 50, Sunday/holiday Rs 10 to 40	NMRC fare table

Track distance is approximated from straight-line distances between stations. Two lines measured longer on the official fares and are stretched by fitted factors: the Magenta Line by 1.12 and the Blue Line by 1.02. scripts/fit_track_factors.py shows that these factors place all 377 DMRC fare constraints in their official slab. Cross-operator trips pay the sum of each operator's segment.

> **Verification**
> scripts/verify_fares.py prices 405 official fares on DMRC's own routes and all match; it runs in the test suite and CI. One official answer is deliberately not followed (Phase 1 to Rajiv Chowk on a Sunday), because the planner itself is inconsistent there; the reason is recorded in the script.

5.3 Database

SQLite in WAL mode with foreign keys, migrated in code (user_version 4). Miles are a ledger: the balance is the sum of entries, so redemptions are negative rows and history is never rewritten.

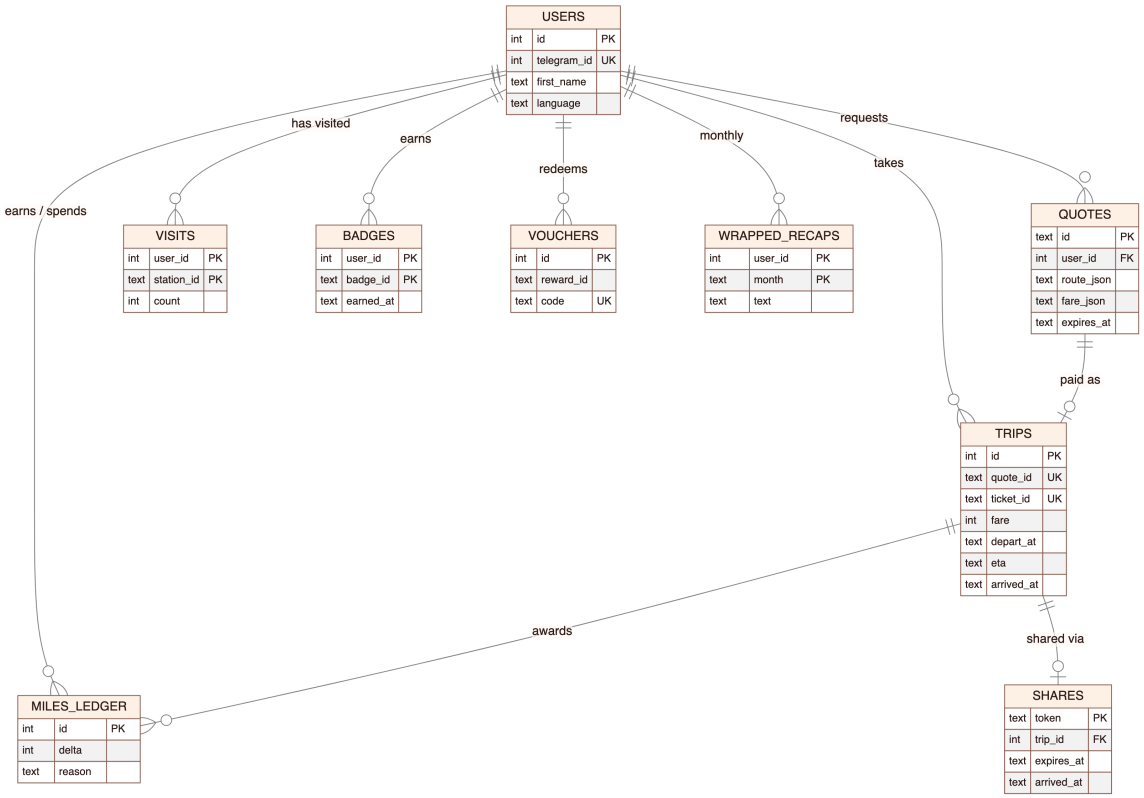


Figure 8. Entity relationships

Table	Purpose
users	Rider, Telegram id, first name, language
quotes	Priced route stored server-side; payments refer to its id; 30-minute expiry
trips	A paid quote: route, fare, departure, ETA, arrival time, ticket id
visits	Stations a rider has started, changed at or finished at, with counts
badges	Badges earned and when
miles_ledger	Every miles award and redemption (balance = sum of deltas)
vouchers	Demo rewards redeemed
wrapped_recaps	Cached monthly narration, refreshed when the month's stats change
shares	Share tokens with expiry and arrival time

6. Domain logic

6.1 Routing

The network is a directed graph whose nodes are (station, line) pairs. Ride edges carry minutes and kilometres; transfers cost six minutes (two between sections of the same line family); walking links use their own times; the Rapid Metro loop is one-way. Fastest uses travel time, fewest changes adds a large penalty per change, and cheapest prices up to eight candidate routes (with and without the Airport Express and footbridge walks) and picks the lowest fare.

6.2 Metro Miles and rewards

Rule	Value
Base	1 mile per km travelled
Off-peak	2x base outside weekday peaks (08:00-11:00, 17:00-20:00); weekends are off-peak
First visit	+25 per station visited for the first time
Streak	+10 from the third consecutive day with a trip, once per day
Arrival	+10 when the traveller taps "I've reached" after half the trip time
Rewards (demo)	Free chai 300 · 10% off a ticket 500 · 15% off at a bookstore 800 · free day pass 2,000

The original plan (5 miles per km, +50, +20) gave a typical commuter about five free day passes a month; the rules were recalibrated so a heavy commuter earns roughly one day pass or a few chais a month.

6.3 Badges

Badges are deterministic rules evaluated after every trip: Agla Station (first trip), Explorer and Cartographer (10 and 25 stations), Interchange Goblin, End of the Line, Night Owl, Early Bird, State Hopper (Delhi, Haryana and UP in a week), Regular, Thousand Miler, Safe and Sound, and a Line Collector badge for each line.

6.4 Metro Wrapped

Monthly statistics are computed in code, including "what it means" comparisons such as "like riding the Blue Line end to end 22 times". The Mini App presents them as a three-stop ride (Ride, Places, Rewards) on a route strip in the rider's favourite line colour; the chat receives a share card and a short narration written by Claude in a defined voice and checked by the number guard.

6.5 Journey sharing

Position is estimated from the timetable: the planned departure and each leg's scheduled minutes. Share links expire two hours after the ETA, or 30 minutes after arrival, and show only the first name.

7. Interfaces

7.1 Telegram bot

A persistent button menu (Stats, Miles, Badges, Wrapped, Dashboard, Language) replaces slash commands and never calls the model. The keyboard holds no URLs, because Telegram stores it on the phone and a changed tunnel address would go stale; the dashboard opens from Telegram's menu button, which the server updates.

7.2 Web application

Endpoint	Purpose	Access
GET /app	Dashboard Mini App	Telegram initData or signed link
GET /api/me, /api/map, /api/badges, /api/miles, /api/insights, /api/wrapped	Dashboard data	Authenticated rider
POST /api/redeem	Redeem a demo reward	Authenticated rider
GET /api/wrapped/{month}/card.png	Wrapped share card	Authenticated rider
GET /s/{token}, /s/{token}/status	Public share page and its 30-second status fragment	Anyone with the token, until expiry
GET /healthz	Health check used by the tunnel watchdog	Public

7.3 MCP server (agla-mcp)

A stdio MCP server built with the official MCP Python SDK exposes seven read-only tools: `agla_search_station`, `agla_plan_route`, `agla_get_fare`, `agla_get_stats`, `agla_get_visited_stations`, `agla_get_badges` and `agla_get_miles`. Route and fare tools accept names or ids; rider tools default to the seeded demo rider. It has no authentication and is intended for local use only.

8. AI design

Aspect	Approach
Model	Claude Sonnet 5.5 at low effort through the Anthropic Python SDK, with server-side fallbacks if a reply is refused
Tools	Seven tools: quote_trip, search_station, get_stats, get_visited_stations, get_miles, get_badges, list_rewards. No tool can pay, log a trip, confirm arrival or redeem.
Prompt caching	On the system prompt and tool definitions; per-turn time and language go in a short system note so the cached prefix stays stable
Number guard	Every number in a reply must come from that turn's tool results; trip replies may contain no numbers at all. Failures fall back to templates.
Cost controls	Menu buttons and repeated off-topic chatter skip the model; 10 model-backed messages per minute and 150 per day per rider; at most four tool rounds; six messages of history
Narration	Wrapped recaps use a voice defined in a Claude skill file that both the development tooling and the runtime prompt read

8.1 Evaluation

A 50-case station-parsing evaluation (abbreviations, typos, Hinglish, landmarks, partial names and ambiguous names that must trigger a question) was run through the Message Batches API and scored on the full system.

Model	Accuracy	Median latency	Cost per 1,000 messages
Claude Sonnet 5.5	50/50	3.90 s	$3.62
Claude Haiku 5.5	50/50	3.24 s	$0.19

Both models were equally accurate. The bot stays on Sonnet because its replies had been tested in all three languages; the recorded results support moving to Haiku if volume grows.

9. Security and privacy

Control	Implementation
Secrets	All keys in a git-ignored .env; the repository is public; gitleaks scans the full history in CI with an extra rule for UPI payment addresses; the process ledger redacts keys, tokens, .env values, emails and UPI addresses
Dashboard authentication	Telegram Mini App initData verified by HMAC (24-hour window) or a signed magic link (7 days); the signing secret must be at least 32 characters and the app refuses to start without it
Web hardening	Pinned CDN scripts with Subresource Integrity; tokens removed from the address bar after load; share pages sent with no-index, no-referrer and no-store headers
Money integrity	Amounts only from stored quotes; trip logging is idempotent and owner-checked; redemption and arrival are buttons
Abuse limits	Per-rider rate limits, tool-round cap, expired-quote purge at startup, early-arrival confirmation without miles
Privacy	Share links expire and show first name only; the bot never messages anyone except the rider
Accepted risks	Telegram initData can be replayed within its 24-hour window; agla-mcp has no authentication (local stdio only)

An independent review by a Claude subagent with no build history found 15 issues, including forgeable dashboard links and a fare bug the tests had missed. All were fixed or documented, each with a regression test.

10. Quality engineering

Layer	Coverage
Unit and integration tests	174 pytest cases: routing, fares, quotes, trips, miles, badges, sharing, Wrapped, dashboard API and authentication, LLM loop with a scripted client, MCP over real stdio, layering rules, ledger
Official data checks	All 405 official fares on DMRC's routes; track-factor fit against 377 constraints
Architecture tests	Front ends may import only the tool layer and may not run SQL
Browser checks	Headless Chromium at phone size clicks every dashboard tab in light and dark mode, plays Wrapped, and fails on any console error
Continuous integration	GitHub Actions on every push: ruff, the full test suite and a gitleaks history scan; actions pinned by commit

11. Operations

11.1 Running the system

1. Install dependencies: `uv sync`.
2. Copy `.env.example` to `.env` and fill in the keys.
3. Optionally seed a demo rider: `uv run python -m scripts.seed_demo`.
4. For demos, run `scripts/run.sh` in a terminal; it keeps the machine awake, restarts the app if it exits and logs to `logs/agla.log`.

11.2 Configuration

Variable	Purpose
ANTHROPIC_API_KEY	Claude API access
TELEGRAM_BOT_TOKEN, TELEGRAM_BOT_USERNAME	The bot's credentials and username
UPI_VPA, UPI_PAYEE_NAME	Payee for UPI intent links (kept out of the public repository)
DASHBOARD_SECRET	Signing key for dashboard links, at least 32 random characters
PUBLIC_BASE_URL	Only when self-hosting; set automatically with --tunnel
AGLA_DB, AGLA_MODEL	Optional overrides for the database path and model

11.3 Troubleshooting

Symptom	Likely cause	Action
Dashboard does not open	Tunnel died (for example after sleep)	Wait about 90 seconds for the watchdog, or restart; tap Dashboard beside the text box
Old share link shows nothing	Tunnel address changed on restart	Share the journey again
Bot does not reply	Process stopped or a second copy is polling	Check `logs/agla.log`; run only one instance
"Slow down" message	Per-rider rate limit reached	Wait a minute (or until the next day for the daily cap)

12. How it was built

The system was built in Claude Code (Claude Opus 5.5) following a phased plan, with the human making every product decision. Over 21.3 hours (7.9 active) the human sent 131 prompts, 43 of them one- or two-word confirmations, and Claude took 876 actions. The process was recorded as it happened.

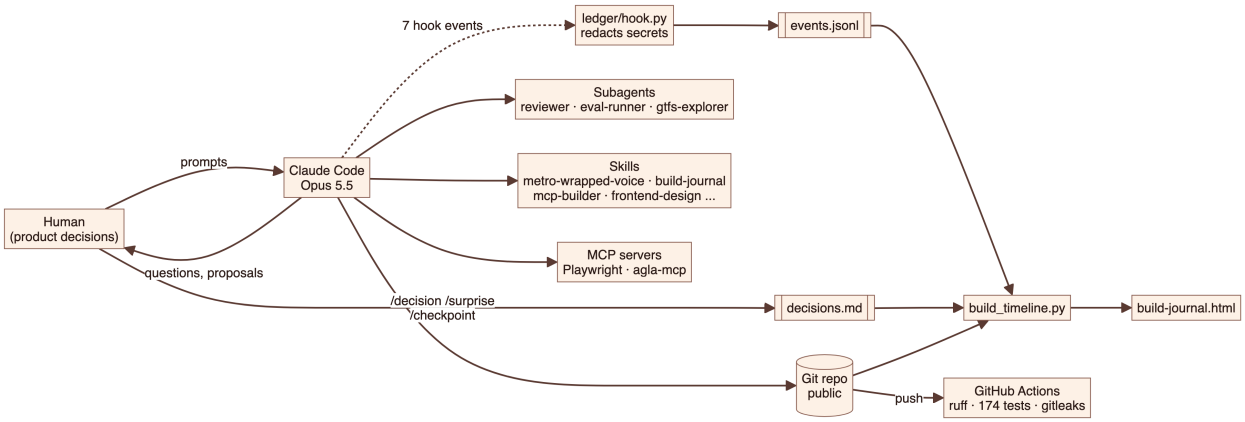


Figure 9. Development process: Claude Code, the process ledger and CI

Claude capability	How it was used
CLAUDE.md	Standing project rules loaded every session (code decides, never invent fares, never commit secrets, ask before architecture changes)
Plan mode	Module interfaces and schema agreed before any code
Hooks and process ledger	Seven hook events append every prompt and tool call, with secrets redacted, to ledger/events.jsonl
Slash commands	/decision, /surprise and /checkpoint record decisions, surprises and commit points in ledger/decisions.md
Subagents	reviewer (independent code audit), eval-runner (model evaluation), gtfs-explorer (data source inspection)
Skills	Created: metro-wrapped-voice, build-journal. Vendored from anthropics/skills: mcp-builder, frontend-design, webapp-testing, skill-creator. Built in: claude-api.
MCP servers	Playwright (browser automation for data snapshots and UI checks) and agla-mcp (the project's own tools)
Plugin	The setup packaged as the installable agla-dev Claude Code plugin

12.1 Logged decisions

Date	Decision	Outcome
23 Oct 2026	Public GitHub repo from the start	git init + public GitHub repo from the start.
23 Oct 2026	Manual spot-check caught a fare bug the automated check missed (surprise)	Fare verification against 384 official DMRC fares showed 97.9% matching, but my manual spot-check of 8 pairs in the DMRC app still caught a wrong fare (Kashmere Gate → Noida City Centre: we said ₹43, DMRC charges ₹54). Our straight-line distance was 20.7 km, just under the 21

Date	Decision	Outcome
		km slab boundary.
23 Oct 2026	Runtime model stays Sonnet 5.5 after the station-parsing eval (override)	Option 3, stay on Sonnet 5.5, and keep the eval results on record.
23 Oct 2026	Metro Miles recalibrated: 1/km, +25 new station, +10 streak	C. 2x off-peak and +10 safe arrival unchanged.
23 Oct 2026	Safety check simplified to "I've reached" + share link (override)	Option 3. Keep 📤 Share journey and a ✅ I've reached button that hands the traveller a ready-made message to send via Telegram's share sheet; remove trusted contacts, invites, timed escalation, the polling job and /demo_fast.
24 Oct 2026	Metro Wrapped as a three-stop metro ride	The three-stop metro ride (d), in the warm marigold-and-gulabi palette across the whole Mini App, with the colours unchanged. Wrapped stays available all the time for the demo; month-end-only access is parked in the backlog.
24 Oct 2026	DMRC fares: shortest distance plus per-line track factors	Option (c): shortest-distance DMRC pricing, track factors Magenta ×1.12 and Blue ×1.02 in fares.yaml (re-checkable with scripts/fit_track_factors.py), a wider "cheapest" search, and the fare check on DMRC's own routes. Result: 384/384 official fares match.

Overrides are decisions where the human chose differently from Claude's suggestion. Many further choices were made through direct questions during the build; docs/build-journal.html shows them on an interactive timeline.

13. Limitations, risks and roadmap

13.1 Known limitations

Limitation	Impact	Mitigation or plan
Mock tickets, unverified payment	Not usable for real travel	Stated in the product; requires DMRC and payment partnerships
Timetable position, not GPS	Share page shows an estimate	Rider can confirm arrival at any time
Approximate track distances	A trip near a slab boundary could be one slab off	Fitted factors match all official fares held; fetch terminal fares to tighten the Blue Line factor
Laptop deployment	Downtime when the host is off; links change on restart	Move to a fixed host and named tunnel
Wrapped lines in English	Hindi and Hinglish riders see the Wrapped story in English	Translate the computed lines

13.2 Backlog

- Location to route: share a Telegram pin or Google Maps link and get the nearest station.
- Follow a friend's journey inside one's own dashboard.
- Metro Wrapped released at month end with a notification.
- Plan stretch goals: group meetups, Paytm staging payments, "next train" estimates.

Appendix A. Glossary

Term	Meaning
DMRC / NMRC	Delhi Metro Rail Corporation / Noida Metro Rail Corporation
Quote	A priced route stored on the server; the payment link and ticket refer to it
Number guard	Check that rejects a model reply containing a number not produced by a tool
Tool layer	agla/tools, the only API through which front ends reach metro logic
Mini App	A web app opened inside Telegram, authenticated by Telegram's signed initData
MCP	Model Context Protocol, a standard way to expose tools to AI clients such as Claude
Quick tunnel	A temporary public HTTPS address from Cloudflare that forwards to the local server
Process ledger	The hook-written log of prompts and tool calls plus the decision log